


# Computacíon Paralela y Distribuída

2025-I

José Fiestas

7 de julio de 2025

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe

# Práctica 04:

# Unidad 4: OpenMP

## PD04: 4 pts (avance presencial)

## Tarea: 3 pts (no incluye avance)

# Ejercicio 1: Escalabilidad de Quicksort (3 pts)

**a)** Paralelice con OpenMP el código adjunto (*qsort.cpp*), tomando en cuenta que la función recursiva qs() puede ser ejecutada en tareas (tasks), dentro de un constructor parallel. Los valores iniciales a ser ordenados serán generados aleatoriamente con valores entre 0 y 99

**b)** Utilice directivas de pre-procesamiento para ejecutar secuencial o paralelo desde el mismo código

**c)** Grafique tiempos de cálculo para N=$10^6$ vs. p=1,2,4,8, ... (¡No olvide dejar de imprimir los arrays en estos casos!)

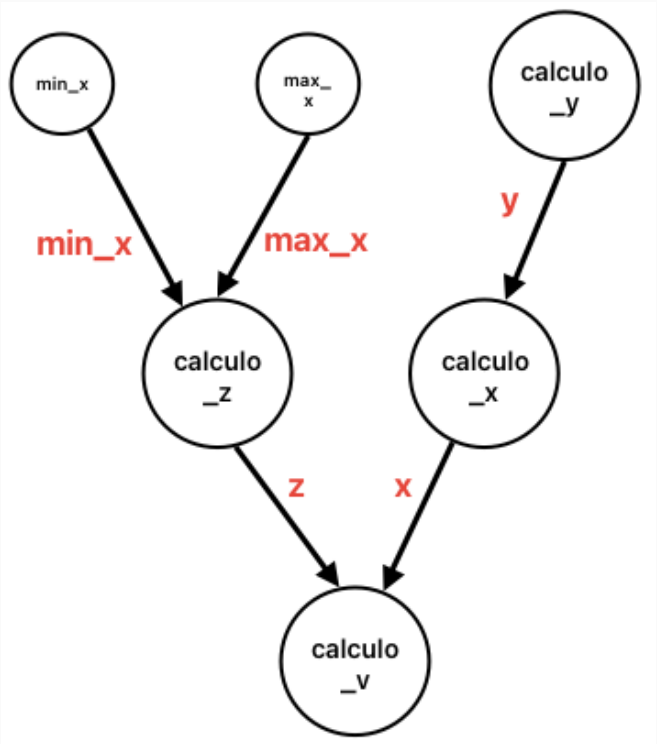
**d)** Compare teoría con experimento superponiendo la curva teórica a la experimental, y analice en que medida se obtiene la complejidad teórica esperada. Si hay diferencias, justifíquelas.

**e)** Haga un análisis cuantitativo del speedup y eficiencia del algoritmo y derive una expresión analítica en función a n y p para estas métricas ¿Se demuestra que Quicksort es escalable?

# Ejercicio 2 OMP, task (2 pts)

Complete y paralelice con OMP la secuencia de funciones mostradas. Defina las funciones de acuerdo a las dependencias mostradas

```
min_x = minimo(x,n);
max_x = maximo(x,n);
calculo_z(z,min_x,max_x,n);
calculo_y(y,x,n);
calculo_x(x,y,n);
calculo_v(v,z,x);
```




Solo el primer argumento de las funciones calculo_() es modificado. Dibuje el DAG para entender mejor el flujo de datos y las dependencias.

Utilice **sections** y **tasks**, y compare las ventajas, desventajas de ambos constructores en este ejercicio.

# Ejercicio 2 OMP, task (2 pts)

**a)** Ver codigos adjuntos: *tasks.cpp*, *sections.cpp*

**b)** En este caso en particular, sections es comparable con tasks en tiempo de ejecución (no muchas tareas en paralelo). Sections es mas simple de programar pero tasks es mas especifica y clara en cuanto las dependencias (lista de variables). Además, la barrera implicita de Sections hace esperar todas las tareas en el bloque, incluso si no tienen dependencias con las siguientes tareas. Esto último es evitado por tasks

# Ejercicio 3: Post-orden (2 pts)

**a)** Paralelice el recorrido post-orden de un árbol binario de n nodos (*tree_postor_sec.cpp*). Utilice omp tasks.

**b)** Utilice directivas de pre-procesamiento (#ifdef), para tener una versión secuencial o paralela según compilación. Manéjelo con un Makefile.

**c)** Mida escalabilidad con un número creciente de nodos.

